

PowerShell Automation Workshop

From Fundamentals to Enterprise-Grade Automation

STUDENT HANDBOOK

Two-Day Hands-On Workshop • 10:00 AM – 6:00 PM ET

Keep this handbook open alongside your console — it has the schedule, key commands, and exercise instructions for both days.

Table of Contents

Welcome & What You'll Learn 3

Before You Arrive 4

Workshop Schedule..... 5

Day 1, Module 1: PowerShell Fundamentals & Environment..... 6

Day 1, Module 2: Cmdlets, Functions & Pipeline Mastery 7

Day 1, Module 3: Scripting & Object Manipulation 8

Day 2, Module 4: Remote Administration & Security 9

Day 2, Module 5: Module Development & Reusability 10

Day 2, Module 6: Cloud Integration & Advanced Automation 11

Quick Reference Cheat Sheet..... 12

Exercise Overview 13

Where to Go Next 14

Welcome & What You'll Learn

Welcome to the PowerShell Automation Workshop! Over the next two days you'll go from PowerShell fundamentals to enterprise-level automation - remote administration, security best practices, reusable modules, and cloud integration with Azure. Every module includes a live demo followed by a hands-on exercise, so come ready to type along and experiment.

By the End of This Workshop, You'll Be Able To...

- Progress from PowerShell fundamentals to enterprise-level automation, including cloud integration and remote administration.
- Apply security best practices, module development, and version control in real-world IT environments.
- Take home a collection of scripts and automation solutions you built yourself and can apply at work immediately.

How This Handbook Works

Each module section below gives you the topics covered, a short cheat sheet of the key commands and concepts, and the hands-on exercise you'll complete. Starter files with TODO comments are in the sample-code.zip archive provided by your instructor - look for the matching filename in each module.

Before You Arrive

Who This Workshop Is For

- IT professionals and system administrators
- DevOps engineers
- Cloud architects
- Automation specialists who manage or support Windows-based systems

Prerequisites

- Basic understanding of Windows administration
- Familiarity with command-line interfaces
- No prior scripting experience required

What to Bring

- Your own computer (Windows or Mac) with a camera, speakers, and a microphone
- A stable internet connection
- PowerShell 5.1 or 7.x already installed

Day 2 Pre-Work

Module 6 uses the Az PowerShell module against a real Azure subscription. Before Day 2, please:

- Create a free-tier Azure account if you don't already have one.
- Install the Az module: `Install-Module -Name Az -Scope CurrentUser -Repository PSGallery`
- Confirm you can sign in: `Connect-AzAccount`

Workshop Schedule

Both days run 10:00 AM – 6:00 PM Eastern Time, with a one-hour lunch break. Exact timing may shift slightly to match the pace of the group.

Day 1 — Foundations

Time (ET)	Segment	Module
10:00	Welcome, introductions, environment check	Setup
10:30	PowerShell Fundamentals & Environment	Module 1
11:55	Break	—
12:10	Cmdlets, Functions & Pipeline Mastery	Module 2
2:10	Lunch	—
3:10	Scripting & Object Manipulation	Module 3
5:15	Break	—
5:30	Recap, Q&A, preview Day 2	Wrap-up

Day 2 — Enterprise Implementation

Time (ET)	Segment	Module
10:00	Day 1 recap / questions	Kickoff
10:10	Remote Administration & Security	Module 4
12:10	Break	—
12:25	Module Development & Reusability	Module 5
2:10	Lunch	—
3:10	Cloud Integration & Advanced Automation	Module 6
5:15	Break	—
5:30	Portfolio review, feedback, wrap-up	Wrap-up

Day 1, Module 1: PowerShell Fundamentals & Environment

Sample code files: 01-EnvironmentCheck.ps1, 02-BasicCommands.ps1

Topics Covered

- The PowerShell ecosystem: engine, host, and editors (ISE, VS Code, Windows Terminal)
- PowerShell 5.1 vs. PowerShell 7.x - what's different and why it matters
- Console navigation, getting help, and running your first commands

Key Commands & Concepts

Command / Concept	What It Does
\$PSVersionTable	Shows your PowerShell version and edition (Desktop vs. Core)
Get-Command	Finds cmdlets, functions, and aliases by name, verb, or noun
Get-Help <cmdlet> - Examples	Shows usage examples for any cmdlet
Get-Verb	Lists the approved verbs used in cmdlet naming (Get-, Set-, New-, ...)
Get-ExecutionPolicy - List	Shows what scripts are currently allowed to run
Get-PSDrive	Lists PowerShell "drives" - filesystem, registry, env vars, and more

Hands-On Exercise

Exercise 1 (starter: Exercise1-EnvironmentCheck.ps1) — write a script that reports your PowerShell version, edition, execution policy, and OS, then prints a clear PASS/WARN line depending on whether your machine meets the PowerShell 5.1+ minimum.

Day 1, Module 2: Cmdlets, Functions & Pipeline Mastery

Sample code files: 01-EssentialCmdlets.ps1, 02-PipelineDemo.ps1, 03-CustomFunctions.ps1

Topics Covered

- Core cmdlets for filtering, sorting, selecting, grouping, and measuring data
- Why the pipeline passes whole objects, not text - and why that matters
- Writing your own functions, from a bare function block up to parameterized, validated, pipeline-aware functions

Key Commands & Concepts

Command / Concept	What It Does
Where-Object	Filters pipeline objects by a condition, e.g. Status -eq 'Running'
Sort-Object	Orders objects by one or more properties
Select-Object	Shapes output to specific properties, including calculated ones
Group-Object / Measure-Object	Groups objects or computes sums/averages/counts
\$_ / \$PSItem	The current object inside a pipeline script block
function Verb-Noun { }	Defines a reusable function - the basic building block, no parameters yet
param(\$Name)	Adds parameters inside the function body so callers can pass in values
param() + [Parameter(Mandatory)]	Makes a parameter required, and adds type/validation attributes
ValueFromPipeline	Lets a function accept piped input directly - makes sense once you've seen the pipeline demo

Hands-On Exercise

Exercise 2 (starter: Exercise2-ResourceReporter.ps1) — using the function-writing pattern from the 03-CustomFunctions.ps1 demo, write Get-TopResourceConsumers, a function that accepts -Type (Process or Service) and -Top <int>, using ValidateSet and calculated properties to shape the output.

Day 1, Module 3: Scripting & Object Manipulation

Sample code files: 01-ControlFlow.ps1, 02-ObjectsAndProperties.ps1, 03-ErrorHandling.ps1

Topics Covered

- Branching logic with if/elseif/else and switch
- Looping with for, foreach, while, and do-while/do-until
- Creating and shaping PSCustomObjects for structured script output
- Converting between PSCustomObject, hashtable, and JSON
- Handling errors gracefully with try/catch/finally

Key Commands & Concepts

Command / Concept	What It Does
if / elseif / else	Runs code branches based on a condition
switch	Matches a value (or each item in a collection) against several cases
for (\$i=0; \$i -lt N; \$i++)	Loops a fixed number of times using a counter
foreach (\$item in \$collection)	Loops over each item in an array already in memory
while (<condition>) { }	Repeats while a condition stays true; may run zero times
do { } while / until (<condition>)	Repeats, but always runs the body at least once
break / continue	Exits a loop entirely / skips to the next iteration
[PSCustomObject]@{...}	Creates a structured object with named properties
ConvertTo-Json / ConvertFrom-Json	Converts objects to/from JSON text
try / catch / finally	Runs code, handles errors, and always runs cleanup code
-ErrorAction Stop / -ErrorVariable	Makes an error catchable / captures it without stopping

Hands-On Exercise

Exercise 3 (starter: Exercise3-BatchFileProcessor.ps1) — write a function that reads a batch of file paths, catches errors per file without stopping the whole batch, and returns a summary object for each file (Path, Success, LineCount, Error).

Day 2, Module 4: Remote Administration & Security

Sample code files: 01-PSRemoting.ps1, 02-SecureCredentials.ps1

Topics Covered

- Running commands on remote machines with Invoke-Command and PSSessions
- The PSCredential / SecureString pattern for handling passwords safely
- Enterprise credential and access patterns: SecretManagement and JEA (awareness level)

Key Commands & Concepts

Command / Concept	What It Does
Invoke-Command - ComputerName	Runs a script block on one or more remote machines
Enter-PSSession / Exit-PSSession	Opens/closes an interactive remote session
New-PSSession / Remove-PSSession	Creates/closes a reusable persistent remote session
Get-Credential	Interactively prompts for a username and password as a PSCredential
Test-WSMan	Checks whether a remote machine is reachable for PowerShell remoting
New-CimSession	Opens a lightweight session for CIM/WMI queries

Hands-On Exercise

Exercise 4 (starter: Exercise4-RemoteServiceCheck.ps1) — use Invoke-Command against the lab servers provided by your instructor, with Get-Credential (never a hardcoded password), to report Spooler and BITS service status from each server.

Day 2, Module 5: Module Development & Reusability

Sample code files: *MyToolkit/MyToolkit.psm1*, *01-CreateModule.ps1*, *02-ModuleManifest.ps1*

Topics Covered

- Structuring functions into an importable .psm1 module
- Generating a module manifest (.psd1) with New-ModuleManifest
- Semantic versioning and Git version-control practices for scripts and modules

Key Commands & Concepts

Command / Concept	What It Does
Export-ModuleMember -Function	Controls which functions in a .psm1 are visible to callers
Import-Module <path> -Force	Loads (or reloads) a module - use -Force while iterating
New-ModuleManifest	Generates a .psd1 manifest file with module metadata
Test-ModuleManifest	Validates and inspects an existing manifest
Get-Command -Module <name>	Lists everything a module exports
MAJOR.MINOR.PATCH	Semantic versioning: breaking / new feature / bug fix

Hands-On Exercise

Exercise 5 (scaffold: *Exercise5-ModuleFolder/*) — package two of your own Day 1 functions into a module, generate a manifest with New-ModuleManifest, and add a CHANGELOG.md entry for version 1.0.0.

Day 2, Module 6: Cloud Integration & Advanced Automation

Sample code files: 01-AzurePowerShellBasics.ps1, 02-EndToEndAutomation.ps1

Topics Covered

- Authenticating to Azure and querying resources with the Az PowerShell module
- Combining remoting, objects, and error handling into one end-to-end automation
- Next steps for production: scheduling, alerting, testing, and versioning

Key Commands & Concepts

Command / Concept	What It Does
Connect-AzAccount	Signs in to Azure interactively
Get-AzSubscription / Set-AzContext	Lists and switches between Azure subscriptions
Get-AzResourceGroup	Lists resource groups in the current subscription
Get-AzVM	Lists virtual machines and their status
New-AzResourceGroup / New-AzVM	Creates a resource group / virtual machine (use with care)
Update-AzTag	Adds or updates governance tags on a resource

Hands-On Exercise

Exercise 6 / Capstone (starter: Exercise6-Capstone.ps1) — extend the end-to-end health-check script with an -UnhealthyOnly filter, a JSON export alongside the CSV export, and one live Az resource query appended to the output.

Quick Reference Cheat Sheet

A condensed, all-in-one lookup of the commands used most across both days.

Discovery & Help

Command / Concept	What It Does
Get-Command	Find cmdlets, functions, and aliases
Get-Help <cmdlet> - Examples	Show usage examples
Get-Member	Show an object's properties and methods
Get-Verb	List approved cmdlet verbs

Control Flow

Command / Concept	What It Does
if / elseif / else	Branch based on a condition
switch	Match a value against multiple cases
for / foreach	Loop a fixed number of times / over each item in an array
while / do-while	Loop while a condition holds / at least once
break / continue	Exit a loop / skip to the next iteration

Filtering, Shaping & Output

Command / Concept	What It Does
Where-Object	Filter objects by a condition
Sort-Object / Group-Object / Measure-Object	Order, group, or summarize objects
Select-Object	Choose or compute specific properties
[PSCustomObject]@{...}	Build a structured object
ConvertTo-Json / Export-Csv	Serialize output to JSON or CSV

Error Handling

Command / Concept	What It Does
try / catch / finally	Handle errors and guarantee cleanup
-ErrorAction Stop	Make an error catchable
-ErrorVariable	Capture errors without stopping

Remoting & Credentials

Command / Concept	What It Does
Invoke-Command - ComputerName	Run a script block remotely
New-PSSession / Enter-PSSession	Open a persistent / interactive remote session
Get-Credential	Prompt for credentials securely

Modules & Cloud

Command / Concept	What It Does
Import-Module / Export-ModuleMember	Load a module / control what it exposes
New-ModuleManifest	Generate a .psd1 manifest
Connect-AzAccount / Get-AzVM	Sign in to Azure / query resources

Exercise Overview

All starter files are in sample-code.zip under the Exercises/ folder. Each has TODO comments guiding you through the task.

Exercise	Starter File	Goal
1	Exercise1-EnvironmentCheck.ps1	Report PS version/edition/execution policy and a PASS/WARN check
2	Exercise2-ResourceReporter.ps1	Build a function reporting top processes or services
3	Exercise3-BatchFileProcessor.ps1	Batch-process files with per-file error handling
4	Exercise4-RemoteServiceCheck.ps1	Check service status across multiple remote servers
5	Exercise5-ModuleFolder/	Package your functions into a versioned module
6	Exercise6-Capstone.ps1	Extend the end-to-end automation with new features

Where to Go Next

PowerShell keeps going well beyond this workshop. A few natural next steps once you're back at your desk:

- `Microsoft.PowerShell.SecretManagement / SecretStore` — store credentials and secrets outside of plaintext files.
- Pester — the standard testing framework for PowerShell scripts and modules; start with the test stub shown in Module 5.
- Just Enough Administration (JEA) — scope down exactly what a remote session is allowed to run.
- Azure Automation Runbooks / Task Scheduler — run your scripts unattended, on a schedule.
- An internal PowerShell Gallery — share modules like the one you built in Module 5 across your team.

Official Documentation

- PowerShell docs: learn.microsoft.com/powershell
- Az PowerShell docs: learn.microsoft.com/powershell/azure
- PowerShell Gallery: powershellgallery.com

Thank you for attending - keep the scripts you built today and in the Day 2 capstone. They're the start of your own automation toolkit.